

the `$wpdb->get_var()` method to retrieve a single value from the database. Simply pass your SQL statement to the method. The SQL that is passed must return only a single field from a single row like this:

```
$post_id = $wpdb->get_var("SELECT ID FROM ' . $wpdb->posts  
 . ' WHERE  
post_author = 1 LIMIT 1");
```

In plain English, this query will retrieve the ID of the most recent post written by the author with an ID of 1. You'll notice that this line of code uses the `$wpdb->posts` property as well. If you recall, this property held the name of the posts table. You'll also notice that the query itself is only retrieving a single field (ID) from a single row (in this case, you added `LIMIT 1` to the query).

Retrieving a column of data

What if you wanted to get all the IDs of the posts by the author with an ID of 1? For this use case, you want to use the `$wpdb->get_col()` method. When only one field will be retrieved, but there will be more than one row of data, you need to use this method.

By default, the data that is returned will be in the form an object; however, you can pass the method a second argument of `ARRAY_A` or `ARRAY_N` to have data returned as an associative or numerical array.

Using the `get_col()` method

```
$object = $wpdb->get_col("SELECT ID FROM ' . $wpdb->posts  
 . ' WHERE post_  
author = 1");  
$assoc_array = $wpdb->get_col("SELECT ID FROM ' . $wpdb->  
>posts . ' WHERE  
post_author = 1", ARRAY_A);  
$num_array = $wpdb->get_col("SELECT ID FROM ' . $wpdb->  
>posts . ' WHERE post_  
author = 1", ARRAY_N);
```

Retrieving a row of data

Similar to `$wpdb->get_col()`, the database class also provides `$wpdb->get_row()`, a method for retrieving only a single row of data. Ignoring the fact that WordPress provides a `get_userdata()` function as an API function to retrieve information about a user, you might use this method to retrieve information about a user with ID 43.